Módulo I5 · Deep learning

Ciencia de Datos e IA Aplicada: de los datos a las decisiones

N2 · Intermedio · Redes neuronales, PyTorch, CNN, regularización y transfer learning

Caso: leer códigos postales escritos a mano

Mapa del módulo

#	Lección
5.1	Del perceptrón a las redes multicapa
5.2	Funciones de activación y de pérdida
5.3	Descenso de gradiente, retropropagación y optimizadores
5.4	PyTorch: tensores, modelos y bucle de entrenamiento
5.5	Redes convolucionales
5.6	Regularización: dropout, early stopping, aumento de datos
5.7	Transfer learning y modelos preentrenados
5.8	Más allá de las imágenes: secuencias, texto y transformers

5.1 Del perceptrón a las redes multicapa

El caso de la semana

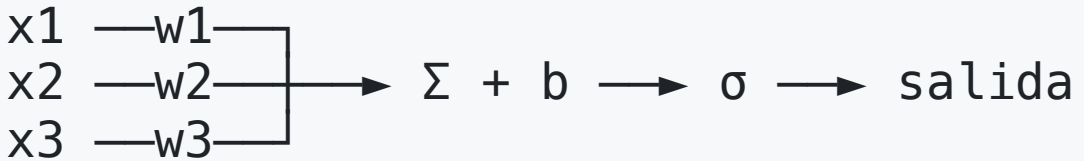
- Empresa de logística: miles de paquetes al día
- Códigos postales **escritos a mano** en la etiqueta
- Objetivo: leerlos automáticamente
- Versión reducida: dígitos de **8×8 píxeles** (1 797 imágenes)

Conjunto	Imágenes
Entrenamiento	1 149
Validación	288
Test	360

La neurona artificial

$$z = w^\top x + b, \qquad \text{salida} = \sigma(z)$$

- Entradas x × **pesos** w + **sesgo** b
- Función σ: escalón (perceptrón, 1958) o sigmoide
- Con sigmoide = **regresión logística** (módulo 04)



El límite de una sola neurona

- Frontera de decisión: $w^\top x + b = 0$ → recta, plano, hiperplano
- Solo separa clases **linealmente separables**
- XOR: "una u otra, pero no las dos" → ninguna recta sirve

x_1	x_2	**XOR**
0	0	0
0	1	1
1	0	1
1	1	0

Capas y perceptrón multicapa (MLP)

$$h = \sigma(Wx + b), \qquad W \in \mathbb{R}^{m \times d}$$

```
entrada (64) → oculta (128) → oculta (64) → salida (10)
```

- **Capas ocultas**: aprenden variables nuevas
- **Capa de salida**: combina esas variables para predecir
- La red hace su propia ingeniería de variables

El MLP del notebook

```
mlp = nn.Sequential(
    nn.Flatten(),                       # 8×8 → 64
    nn.Linear(64, 128), nn.ReLU(),
    nn.Linear(128, 64), nn.ReLU(),
    nn.Linear(64, 10),
)
```

Parámetros por capa: $d \cdot m + m$

$8\,320 + 8\,256 + 650 =$ **17 226** (regresión logística: 650)

¿Por qué funciona?

- **Aproximación universal**: una capa oculta ancha aproxima cualquier función continua
- Pero no dice cuántas neuronas ni si el entrenamiento las encontrará
- Las redes **profundas** reutilizan patrones: trazos → formas → dígitos
- Más capas = más capacidad = más riesgo de sobreajuste

Ideas clave 5.1

Dato	Primera opción
Tabla (clientes, siniestros)	Modelos clásicos (*gradient boosting*)
Imágenes, texto, audio	Deep learning

- Neurona = suma ponderada + sesgo + función
- Apilar capas **con no linealidad** → fronteras arbitrarias
- Cuenta los parámetros antes de entrenar

5.2 Funciones de activación y de pérdida

Sin no linealidad, la profundidad no sirve

$$\hat{y} = W_2(W_1 x + b_1) + b_2 = \underbrace{W_2 W_1}_{W'} x + \underbrace{W_2 b_1 + b_2}_{b'}$$

- Dos capas lineales = **una** capa lineal
- 200 capas lineales = **una** capa lineal
- La activación rompe la linealidad

Activaciones habituales

Activación	Fórmula	Rango	Uso
ReLU	$\max(0, z)$	$[0, \infty)$	Capas ocultas
Sigmoide	$1/(1+e^{-z})$	$(0, 1)$	Salida binaria
Tanh	$\tanh z$	$(-1, 1)$	Redes recurrentes
Softmax	$e^{z_k} / \sum_j e^{z_j}$	suman 1	Salida multiclase

Saturación y ReLU

- Sigmoide y tanh se **aplanan** si $|z|$ es grande → pendiente ≈ 0
- Gradientes que se multiplican y **se desvanecen** en redes profundas
- ReLU: pendiente 1 si $z > 0$, barata, funciona bien
- Riesgo: neuronas "muertas" → Leaky ReLU, GELU
- Regla práctica: **ReLU en capas ocultas**

La capa de salida depende del problema

Problema	Salida	Pérdida
Regresión (unidades vendidas)	Lineal	Error cuadrático medio
Binaria (fraude sí/no)	1 neurona + sigmoide	Entropía cruzada binaria
Multiclase (dígito 0–9)	K logits + softmax	Entropía cruzada

Entropía cruzada

$$L = -\frac{1}{n}\sum_{i} \log p_{i,\, y_i}$$

Probabilidad a la clase correcta	Pérdida
0,9	0,105
0,5	0,693
0,1	2,303

- Castiga mucho la **confianza equivocada**
- Modelo recién inicializado, 10 clases: $\ln 10 \approx 2{,}30$

En PyTorch: logits

```
perdida_fn = nn.CrossEntropyLoss()      # incluye softmax + log
logits = modelo(xb)                     # (lote, 10), SIN softmax
perdida = perdida_fn(logits, yb)        # yb: enteros (long)
probs = logits.softmax(dim=1)           # solo para interpretar
```

- No apliques softmax antes de `CrossEntropyLoss`
- `weight=` para dar más peso a clases costosas o raras

Ideas clave 5.2

- Sin activación no lineal, apilar capas no aporta nada
- ReLU en ocultas; sigmoide/softmax en la salida
- La pérdida traduce "qué es un error" a un número
- Pérdida inicial muy distinta de $\ln K$ → revisa datos o salida
- MSE para regresión, entropía cruzada para clasificar

5.3 Descenso de gradiente, retropropagación y optimizadores

Bajar la montaña con niebla

- La montaña: la pérdida $L(\theta)$
- θ: **todos** los pesos y sesgos (17 226 en el MLP)
- La pendiente: el **gradiente** $\nabla_\theta L$
- Estrategia: paso cuesta abajo, volver a medir, repetir

La regla de actualización

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$$

- η: **tasa de aprendizaje** (tamaño del paso)
- Demasiado pequeña → lentísimo
- Demasiado grande → rebota o diverge
- El hiperparámetro más importante

Ejercicio 3: la tasa de aprendizaje importa

MLP con Adam, 40 épocas (notebook de soluciones):

η	**Pérdida train final**	**Exactitud test**	**Curva**
10^{-4}	0,54	0,911	Lenta, sin converger
10^{-3}	0,03	0,967	Rápida y estable
10^{-2}	≈ 0	0,969	Rápida, con repuntes
10^{-1}	0,35	0,842	Atascada

Minilotes, épocas e iteraciones

- **Lote** (*batch*): 64 imágenes → un gradiente → una actualización
- **Época**: una pasada por todo el entrenamiento
- Notebook: $\lceil 1149/64 \rceil = 18$ **iteraciones por época**
- El ruido de los lotes ayuda a escapar de zonas planas

Retropropagación = regla de la cadena

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

```
forward:   x → z → ŷ → L     (guardar valores)
backward:  x ← z ← ŷ ← L     (multiplicar derivadas)
```

- Coste del *backward* ≈ coste del *forward*
- PyTorch lo hace solo: `perdida.backward()`

Un ejemplo con números

$z = wx + b, \hat{y} = \sigma(z), L = (\hat{y} - y)^2$

Con $x = 1, w = 0{,}5, b = 0, y = 1$:

- $z = 0{,}5, \hat{y} \approx 0{,}62, \sigma'(z) = \hat{y}(1 - \hat{y}) \approx 0{,}235$
- $\partial L / \partial w \approx 2(0{,}62 - 1)(0{,}235)(1) \approx -0{,}18$
- Gradiente negativo → subir w reduce la pérdida

Optimizadores

Optimizador	Idea
SGD	Paso básico; zigzaguea en valles alargados
SGD + momento	Inercia: media de gradientes recientes
Adam	Momento + tasa adaptativa por parámetro
AdamW	Adam + *weight decay* bien aplicado

Punto de partida: `torch.optim.Adam(modelo.parameters(), lr=1e-3)`

Ideas clave 5.3

- $\theta \leftarrow \theta - \eta \nabla L$: pasos cuesta abajo
- Explora η en factores de 10 y **mira la curva**
- `backward()` calcula gradientes; `step()` actualiza
- Retropropagación no aprende: calcula. Aprende el optimizador
- Compara configuraciones con varias semillas

5.4 PyTorch: tensores, modelos y bucle de entrenamiento

¿Por qué PyTorch?

- **Tensores**: como arrays de NumPy, pero también en GPU
- **Autograd**: gradientes automáticos
- **Caja de herramientas**: capas, pérdidas, optimizadores, datos
- Estilo "Python normal": el bucle lo escribes tú
- Más trabajo que `.fit()`, pero control total

Tensores: forma, tipo, dispositivo

```
x = torch.tensor(X_train).unsqueeze(1)
x.shape    # (1149, 1, 8, 8) → (lote, canales, alto, ancho)
x.dtype    # torch.float32
x.device   # cpu  (o cuda:0)
```

- Etiquetas: enteros `int64` para `CrossEntropyLoss`
- Casi todos los errores: **forma, tipo o dispositivo**

Autograd en tres líneas

```
w = torch.tensor(2.0, requires_grad=True)
L = (3 * w - 5) ** 2
L.backward()
print(w.grad)   # 2·(3w−5)·3 = 6
```

- PyTorch registra las operaciones y recorre el grafo hacia atrás
- Los parámetros de `nn.Linear`, `nn.Conv2d` … ya lo tienen activado

Modelos: `nn.Module`

```
class MiMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.oculta = nn.Linear(64, 128)
        self.salida = nn.Linear(128, 10)
    def forward(self, x):
        return self.salida(torch.relu(self.oculta(x.flatten(1))))
```

Cadenas simples: `nn.Sequential(...)` · Parámetros: `sum(p.numel() for p in m.parameters())`

Datos: `Dataset` y `DataLoader`

```
def loader(X, y, batch=64, shuffle=False):
    ds = TensorDataset(torch.tensor(X).unsqueeze(1), torch.tensor(y))
    return DataLoader(ds, batch_size=batch, shuffle=shuffle)
```

- `shuffle=True` solo en entrenamiento
- Datos grandes en disco → `Dataset` propio que lee bajo demanda

El bucle: cinco pasos

```
for xb, yb in dl_train:
    opt.zero_grad()                      # 1. borrar gradientes
    logits = modelo(xb)                  # 2. forward
    perdida = perdida_fn(logits, yb)     # 3. pérdida
    perdida.backward()                   # 4. retropropagación
    opt.step()                           # 5. actualizar
```

Alrededor del bucle

Pieza	Para qué
`modelo.train()` / `modelo.eval()`	Dropout y BatchNorm cambian de modo
`torch.no_grad()`	Evaluar sin construir el grafo
Historial por época	Curvas de aprendizaje
`paciencia` + `state_dict`	Early stopping con la mejor época
`.to(dispositivo)`	CPU o GPU
`torch.save(m.state_dict(), ...)`	Guardar pesos

Errores frecuentes

- Olvidar `zero_grad()` → gradientes acumulados
- Evaluar sin `eval()` → dropout activo, métricas peores
- Olvidar `Flatten` → formas incompatibles
- Preprocesado distinto en producción (0–255 frente a 0–1)
- Usar el test para decidir → solo al final

5.5 Redes convolucionales

El problema de aplanar

- `nn.Flatten()`: la imagen 8×8 → vector de 64
- El MLP **no sabe** qué píxeles son vecinos
- Foto real 224×224×3 = 150 528 valores
- Una capa densa de 1 000 neuronas → **150 millones** de pesos

La convolución: una lupa que recorre la imagen

$$\text{salida}(i,j) = \sum_{u,v} w_{u,v}\, x_{i+u,\, j+v} + b$$

```
imagen              filtro 3×3          mapa de características
. . . . . . .       −1  0  +1           responde fuerte donde
. . # # . . ∗       −1  0  +1   ⟶       hay un borde vertical
. . # # . .         −1  0  +1
```

Los pesos del filtro **se aprenden**

Dos ideas que lo explican todo

1. **Conectividad local**: cada salida mira una vecindad pequeña
2. **Pesos compartidos**: el mismo filtro en todas las posiciones

Consecuencias:

- Muchos menos parámetros
- El patrón se detecta **esté donde esté**

Canales, relleno, paso y pooling

Concepto	Qué hace
Canales	16 filtros → 16 mapas; la siguiente capa los combina
Relleno (*padding*)	Marco de ceros; `padding=1` conserva 8×8
Paso (*stride*)	Cuánto salta el filtro
`MaxPool2d(2)`	Máximo de cada 2×2: mitad de alto y ancho

$$n_{\text{salida}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

La CNN del notebook

Capa	Salida	Parámetros
`Conv2d(1,16,3,padding=1)` + ReLU	16×8×8	160
`MaxPool2d(2)`	16×4×4	0
`Conv2d(16,32,3,padding=1)` + ReLU	32×4×4	4 640
`MaxPool2d(2)`	32×2×2	0
`Flatten` + `Dropout(0.3)`	128	0
`Linear(128,10)`	10	1 290
Total		**6 090**

Resultados del notebook

Modelo	Parámetros	Exactitud test
Regresión logística	650	0,964
MLP	17 226	0,967
CNN	6 090	0,969

- CNN: un tercio de parámetros que el MLP, igual o mejor
- Con 8×8 limpias la diferencia es pequeña; con fotos reales, enorme

Jerarquía de características

```
capa 1: bordes, trazos → capa 2: curvas, esquinas → capas finales: dígitos, objetos
```

- Campo receptivo cada vez mayor
- 2012: AlexNet gana ImageNet con una CNN
- VGG, ResNet (conexiones residuales, >100 capas)
- Sección 7 del notebook: los 16 filtros 3×3 aprendidos

Errores frecuentes

- Olvidar la dimensión de canales (N, C, H, W)
- Calcular mal el tamaño antes de la capa densa → imprime `x.shape`
- Suponer invariancia a rotaciones o escalas que no ha visto
- Usar CNN en datos tabulares: las columnas no son "vecinas"

5.6 Regularización: dropout, early stopping, aumento de datos

El estudiante que memoriza

- Decenas de miles de parámetros, ~1 000 imágenes
- Puede **memorizar** el entrenamiento: 100 % de acierto
- Y fallar más de lo debido en lo nuevo
- Mismo sobreajuste que en el módulo 03, con más capacidad

Diagnóstico con curvas de aprendizaje

Lo que ves	Diagnóstico	Acción
Train y validación malas	Infraajuste	Más capacidad, más épocas, revisar η
Train excelente, validación empeora	Sobreajuste	Regularizar, más datos, parar antes
Ambas buenas y cercanas	Buen ajuste	Evaluar en test

Ejercicio 2: MLP ×2, 150 épocas

- 50 826 parámetros, sin early stopping
- Exactitud train: **1,000**; pérdida train ≈ 0,0003
- Pérdida validación: mínima **0,145** (época 16) → **0,229** al final
- Exactitud validación ≈ 0,97 (estable)
- Sobreajuste "suave": el modelo se vuelve **demasiado confiado**

Dropout

```
entrenamiento:  ●  ○  ●  ●  ○  ●   (○ = apagada al azar, p = 0,3)
evaluación:     ●  ●  ●  ●  ●  ●   (todas activas)
```

- La red no puede depender de una neurona concreta
- Como promediar muchas subredes
- `nn.Dropout(0.3)` ; se desactiva con `modelo.eval()`

Weight decay y early stopping

- **Weight decay** (L2, como Ridge): $L + \lambda \sum \theta_j^2$
  `torch.optim.AdamW(..., weight_decay=1e-2)`
- **Early stopping**: parar tras `paciencia` épocas sin mejorar
- Restaurar el **mejor** `state_dict`
- La más barata: también ahorra cómputo
- El número que se reporta: el de **test**

Aumento de datos

- Variaciones realistas con la **misma etiqueta**
- Desplazar, girar un poco, brillo, recortes
- Cuidado: un 6 girado 180° es un 9
- Solo en **entrenamiento**, nunca en validación ni test
- Ejercicio 4: `np.roll` ±1 píxel, nuevo en cada época

Ejercicio 4: resultados (media de 3 semillas)

CNN	Test original	Test desplazado ±1 px
Sin aumento	0,969	0,550
Con aumento	0,967	**0,907**

- En el test original: **sin mejora**
- Ante dígitos descentrados: robustez enorme
- Con una sola semilla habríamos "visto" una mejora falsa

Ideas clave 5.6

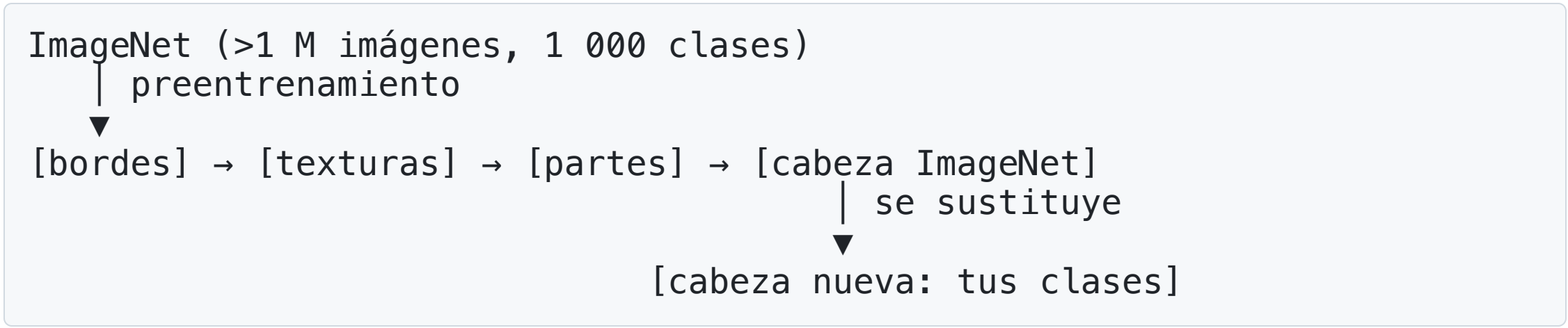
- Curvas train frente a validación: el diagnóstico básico
- Dropout, weight decay, early stopping, aumento de datos
- La pérdida de validación avisa antes que la exactitud
- El aumento enseña las **invariancias** de producción
- Con 360 imágenes de test, cada error = 0,28 puntos

5.7 Transfer learning y modelos preentrenados

Pocos datos, mucha ambición

- 500 fotos de productos defectuosos
- Una CNN moderna: millones de parámetros
- Desde cero → sobreajuste seguro
- Solución: reutilizar lo que otra red ya aprendió

Las primeras capas son universales



Dos estrategias

	Extractor de características	**Ajuste fino**
Pesos preentrenados	Congelados	Se siguen entrenando
Qué se entrena	Solo la cabeza	Cabeza + últimas capas (o todo)
Tasa de aprendizaje	Normal	**Pequeña**
Datos necesarios	Pocos	Algo más
Riesgo de sobreajuste	Bajo	Moderado

En código (torchvision)

```
from torchvision import models
red = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in red.parameters():
    p.requires_grad = False                    # congelar
red.fc = nn.Linear(red.fc.in_features, 2)     # nueva cabeza
opt = torch.optim.Adam(red.fc.parameters(), lr=1e-3)
```

- Preprocesa **igual** que en el preentrenamiento
- Combínalo con aumento de datos

¿Qué estrategia?

Situación	Recomendación
Pocos datos, fotos "normales"	Extractor de características
Miles de datos, dominio algo distinto	Ajuste fino de últimas capas
Muchos datos, dominio muy distinto	Ajuste fino completo o desde cero
Datos tabulares	Modelos clásicos

¿Cuándo compensa el deep learning?

- **Tipo de dato**: no estructurado → sí; tabla → empieza por lo clásico
- **Datos etiquetados**: pocos → solo con transferencia
- **Coste y operación**: cómputo, preprocesado idéntico, interpretabilidad
- **Línea base primero**: logística 0,964 frente a CNN 0,969 en 8×8
- Modelos fundacionales: BERT, GPT, CLIP (módulo 09)

Ideas clave 5.7

- Red preentrenada = detectores genéricos reutilizables
- Extractor: congelar y entrenar la cabeza
- Ajuste fino: seguir entrenando con tasa pequeña
- Vigila la **licencia** de los pesos y el sesgo del preentrenamiento
- Valida con datos de otro periodo o zona

5.8 Más allá de las imágenes: secuencias, texto y transformers

Datos con orden

- Ventas diarias, clics de una sesión, palabras de un correo, sensores
- El **orden** importa y la **longitud** varía
- Un MLP espera un vector de tamaño fijo

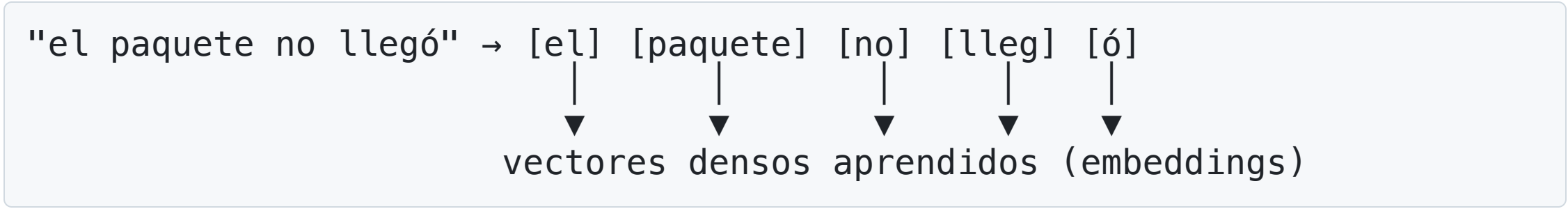
Redes recurrentes

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$



- Estado oculto = resumen de lo visto
- Gradientes que se desvanecen → **LSTM**, **GRU** (puertas)
- También: convoluciones 1D

Texto: tokens y embeddings



- Palabras con usos parecidos → vectores cercanos
- "factura" ≈ "recibo"

Atención y transformers

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- Cada token **consulta** a todos los demás
- "la dirección **estaba** mal" → "estaba" mira a "dirección"
- Procesamiento **en paralelo** (frente a la RNN, paso a paso)
- *Attention Is All You Need* (2017)

Dos familias

	Codificador (BERT)	Decodificador (GPT)
Lee	Texto completo	De izquierda a derecha
Para	Clasificar, extraer, buscar	Generar texto
Ejemplo	Enrutar correos por motivo	Redactar respuestas

Vision Transformers: imágenes troceadas en parches = tokens

¿Qué usar en la práctica?

Problema	Primera opción
Clasificar correos o reseñas	Transformer preentrenado o LLM vía API
Previsión de demanda	Series temporales clásicas (módulo 08)
Voz a texto	Modelo preentrenado
Anomalías en sensores	Estadística primero; redes si hay muchos datos

Cierre del módulo

- Neurona → MLP → CNN → transformers: las mismas piezas
- Capas + activaciones + pérdida + gradientes
- Regulariza y valida con cuidado
- Casi siempre: **parte de un modelo preentrenado**
- Siguiente parada: sistemas de recomendación (módulo 06)